FallGuard AI

Spatiotemporal Deep Learning Fall Detection & Privacy Preservation System

Domain: Computer Vision, Deep Learning, Healthcare IoT

Core Framework: PyTorch, OpenCV 5.0, Scikit-Learn, Streamlit

Primary Models: C3D (weights.h5, 78.4M) + YuNet ONNX Face Shield

Secondary Model: PicoDet + ByteTrack + RTMPose + SGN Skeleton

1. High-Level Executive Summary

FallGuard AI is an automated, privacy-preserving video monitoring system tailored for elderly care facilities, hospitals, and assisted-living environments. Falls represent the leading cause of accidental injury and mortality in older adults. Conventional wearable alarms often fail due to poor compliance (dementia, forgetting to wear them, or battery drain), while standard CCTV cameras severely breach human dignity in bedrooms and private quarters.

FallGuard AI solves both dilemmas simultaneously through a dual-branch neural architecture:

1. **Spatiotemporal Action Recognition:** Evaluates temporal velocity and posture shifts across 16-frame rolling video buffers using a 3D-CNN (C3D) and calibrated binary classification head.
2. **Autonomous Privacy Shield:** Employs OpenCV 5.0 YuNet DNN to detect faces in real-time, enforcing irreversible privacy filters (Gaussian, Mosaic, Solid Blackout, Frosted Glass) with custom safety boundary padding before video is stored or shown.
3. **Fall Kinematics Classification:** Uses dense Farneback optical flow to differentiate forward, backward, sideways, and sudden collapse dynamics without requiring invasive sensors.

2. End-to-End System Architectural Flow



3. Data Cleaning & Preprocessing Flow (Step-by-Step)

Step 1 **Video Decoding & Frame Ingestion**

Frames are ingested sequentially using OpenCV's `cv2.VideoCapture`. Hardware timestamps, native frame rate (FPS), and frame dimensions are parsed. Raw video frames arrive in BGR format.

Code Location: `C:\Users\Jay\AIML_FallDetection_jay\video_pipeline.py` (Lines 120-155)

Step 2 **Color Space Conversion (BGR to RGB)**

OpenCV's BGR format is converted to RGB via `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`. This is required because pre-trained C3D convolution kernels and web browsers expect RGB channel ordering.

Code Location: `C:\Users\Jay\AIML_FallDetection_jay\video_pipeline.py` (Lines 160-175)

Step 3 **Spatial Downsampling to 112 x 112 Pixels**

Frames are resized to 112×112 pixels using bilinear interpolation (`cv2.INTER_LINEAR`). This exact resolution satisfies the spatial dimension constraints of C3D's 5 max-pooling stages ($112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7 \rightarrow 4$). Any other resolution causes dimension mismatches at the FC6 linear layer.

Code Location: `C:\Users\Jay\AIML_FallDetection_jay\video_pipeline.py` (Lines 180-195)

Step 4 **Temporal FIFO Rolling Window Buffer (16 Frames)**

A First-In-First-Out (FIFO) queue keeps exactly 16 consecutive frames. When frame t arrives, it is appended; once the length reaches 17, frame $t-16$ is popped. This ensures $O(1)$ constant memory usage (~600 KB RAM) without leaking memory during continuous 24/7 monitoring.

Code Location: `C:\Users\Jay\AIML_FallDetection_jay\video_pipeline.py` (Lines 200-225)

Step 5 **Channel-Wise Mean Subtraction Normalization**

The buffer is converted to a `float32` array and centered using Sports-1M pretraining channel means: `clip_arr = clip_arr - np.array([124.0, 117.0, 104.0], dtype=np.float32)`. Centering values around zero accelerates feature convergence and matches original C3D activation statistics.

Code Location: `C:\Users\Jay\AIML_FallDetection_jay\video_pipeline.py` (Lines 230-245)

Step 6 **Tensor Dimensional Permutation**

PyTorch requires channels-first 3D convolutional format: `(Batch, Channels, Time, Height, Width)`. The array of shape `(16, 112, 112, 3)` is transposed to `(3, 16, 112, 112)` and unsqueezed to `(1, 3, 16, 112, 112)`.

Code Location: `C:\Users\Jay\AIML_FallDetection_jay\c3d_model.py` (Lines 80-100)

Step 7 **Dataset Metadata Interval Cleaning & Parsing**

Ground truth CSV annotations in GMDCSA24 (e.g. `"Falling (SW)[3.4 to 6]; Sitting[0 to 3.4]"`) are parsed with regular expressions into clean timestamps and boolean fall flags for verification.

Code Location: `C:\Users\Jay\AIML_FallDetection_jay\dataset_manager.py` (Lines 6-35)

4. Core Algorithms & Technologies (Primary Pipeline)

Technology / Algorithm	What It Does & Mathematical Role	File Path in Project
C3D 3D-CNN Backbone 78.4M Params	Performs spatiotemporal feature extraction across space and time simultaneously. Traverses 8 Conv3D and 5 MaxPool3D layers to output a dense 4,096-dimensional FC7 embedding vector representing human action dynamics.	`C:\Users\Jay\AIML_FallDetection_jay\c3d_model.py` `C:\Users\Jay\AIML_FallDetection_jay\weights.h5`
Calibrated Logistic Fall Head Supervised ML	Regularized Logistic Regression ($C=0.5$, balanced class weighting) trained on GMDCSA24 embeddings. Applies sigmoid mapping $P(\text{Fall}) = 1 / (1 + \exp(-(w^T x + b)))$ to produce clinical probabilities.	`C:\Users\Jay\AIML_FallDetection_jay\train_head.py` `C:\Users\Jay\AIML_FallDetection_jay\models\fall_head.joblib`
YuNet ONNX Face Shield Privacy DNN	Ultra-lightweight edge face detector operating on native resolution frames. Locates facial landmarks, expands bounding boxes by 20%-25% margin padding, and irreversibly renders Gaussian, Mosaic, Blackout, or Frosted Glass filters.	`C:\Users\Jay\AIML_FallDetection_jay\face_blur.py` `C:\Users\Jay\AIML_FallDetection_jay\models\face_detection_yunet.onnx`
Farneback Optical Flow Motion Kinematics	Calculates dense 2D motion displacement fields (dx, dy) across consecutive frames inside the fall onset window to categorize fall dynamics: Forward, Backward, Sideways, Sudden Collapse, or Loss of Balance.	`C:\Users\Jay\AIML_FallDetection_jay\video_pipeline.py` (Class `FallSubtypeClassifier`)
FFmpeg H.264 Transcoder Web Video	Converts raw OpenCV `mp4v` video streams into web-native H.264 (`libx264`, `yuv420p` pixel format) with `-movflags faststart` for zero-latency browser rendering.	`C:\Users\Jay\AIML_FallDetection_jay\video_pipeline.py` (Function `reencode_h264`)

5. Secondary / Pose-Based Pipeline (in `Jay/` Folder)

Your repository includes a secondary skeleton/pose-based fall detection system located in the `C:\Users\Jay\AIML_FallDetection_jay\Jay\` directory. This pipeline detects human joints and recognizes falls via graph neural networks:

Component	Function & Underlying Algorithm	File Path
PicoDet Detector	Ultra-lightweight anchor-free object detector (`pico.onnx`). Detects human bounding boxes at 100+ FPS on edge CPUs.	`C:\Users\Jay\AIML_FallDetection_jay\Jay\PicoDet.py` `C:\Users\Jay\AIML_FallDetection_jay\Jay\pico.onnx`
ByteTrack Multi-Tracker	Tracking-by-detection framework using Kalman filter velocity state estimation to maintain track IDs for multiple people across frames.	`C:\Users\Jay\AIML_FallDetection_jay\Jay\ByteTracker\byte_tracker.py` `C:\Users\Jay\AIML_FallDetection_jay\Jay\ByteTracker\kalman_filter.py`
RTMPose Estimator	2D multi-person pose estimation (`rtmpose-small.onnx`) using SimCC (Simple Coordinate Classification) to extract 17 keypoint coordinates.	`C:\Users\Jay\AIML_FallDetection_jay\Jay\rtmpose-small.onnx`
SGN Backbone	Semantics-Guided Neural Network modeling spatial joint-level correlations and temporal frame-level dynamics for skeleton action recognition.	`C:\Users\Jay\AIML_FallDetection_jay\Jay\models\SGN\backbone.py`
SGN Checkpoint	Domain-Adapted trained PyTorch weights classifying skeleton sequences into *Normal* vs. *Fall*.	`C:\Users\Jay\AIML_FallDetection_jay\Jay\fall_detect944-DA.pth` `C:\Users\Jay\AIML_FallDetection_jay\Jay\label.txt`

6. Evaluation Suites & Web Dashboard

Evaluation Notebooks & Colab Scripts

- **Comprehensive Evaluation Notebook:** `C:\Users\Jay\AIML_FallDetection_jay\evaluate_model_performance.ipynb`
  Performs 28-metric clinical benchmarking (Accuracy, Recall, Specificity, F1, F2-score, MCC, Brier score, ROC-AUC, PR-AUC, and Leave-One-Subject-Out cross-validation across Subjects 1–4).
- **Google Colab Runner:** `C:\Users\Jay\AIML_FallDetection_jay\colab_evaluate.py`
  Complete script configured for Google Drive mounting and GPU execution with automated feature serialization to `dataset_features.npz`.

Interactive Web Application (Streamlit)

Implemented in `C:\Users\Jay\AIML_FallDetection_jay\app.py` featuring 5 specialized tabs:

1. **Tab 1: Video Analysis & Live Telemetry:** Real-time dual video player (raw vs. anonymized), risk gauges, and Plotly confidence timelines.
2. **Tab 2: Face Anonymization Lab:** 4-way visual comparison workbench for Gaussian, Mosaic, Blackout, and Frosted Glass filters.
3. **Tab 3: C3D Model Architecture:** Layer-by-layer parameter inspector and weight distribution histograms.
4. **Tab 4: GMDCSA24 Dataset Explorer:** Filterable catalog across all 160 video clips and duration analytics.
5. **Tab 5: Compliance & Incident Audit:** Exportable GDPR/HIPAA audit certificates recording timestamps and peak risk values.

FallGuard AI © 2026. Generated for academic defense, viva examination, and technical reference.